

Assignment

Building a RESTful Book Information Service with Spring Boot and JSON

GenAI Level 1 — Limited AI Use ⚠

This assignment is classified as **Level 1 — Limited AI Use**. Please follow the **Level 1 GenAI requirements** described in the **Generative AI Use in Assessments policy** posted at the top of the **Assignments** section in Moodle.

1. Scenario

You have joined a company that operates an **online bookstore inventory system**. The company maintains a catalog of books in a JSON data file. Each book has a unique ID along with information such as its title, author, publication year, category, and price.

The company wants to expose this information through a simple **RESTful web service** so that other applications can retrieve book information using HTTP requests.

For this assignment, you will build a Spring Boot application that:

- loads book information from a provided JSON file;
- converts the JSON data into Java objects;
- stores the books in memory while the application is running;
- provides a REST endpoint for retrieving all books;
- provides another REST endpoint for retrieving a specific book by its ID;
- returns appropriate HTTP responses when a requested book does not exist.

The **books.json** file required for this assignment will be provided as an **attachment on Moodle**. You should use the provided file rather than creating your own dataset.

2. Learning Objectives

By completing this assignment, you will practice:

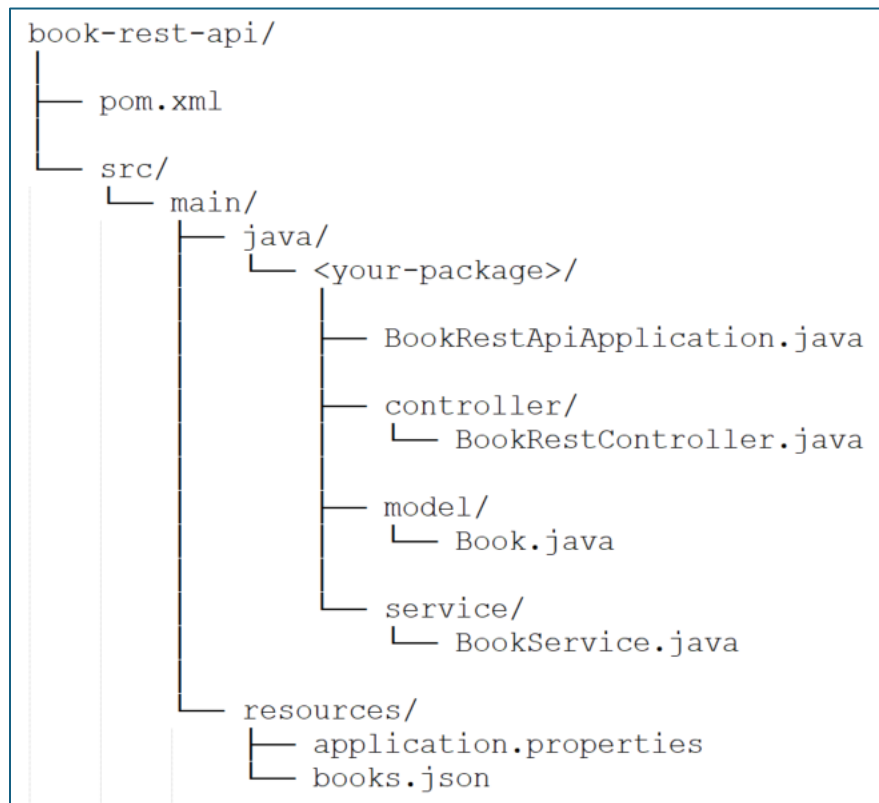
- creating a Spring Boot REST application;
- working with JSON data;
- mapping JSON objects to Java objects;
- using a layered application structure;
- using Spring's `@RestController`;

- creating REST endpoints with @GetMapping;
- extracting information from URLs using @PathVariable;
- implementing basic search logic using a book ID;
- returning appropriate HTTP status codes;
- testing REST endpoints using a browser and/or Postman.

3. Required Application Structure

Create a **Maven-based Spring Boot project** using Spring Initializr.

Your application should follow approximately this structure:



You may use a different root package name, but the application should maintain a clear separation between the **model**, **service**, and **controller** responsibilities.

Note

The structure above specifies what components your application should contain. You are responsible for implementing the classes and methods yourself.

4. Part A — Create the Spring Boot Project

Use **Spring Initializr** to create a new Spring Boot application.

Configure it as a:

- Maven project;
- Java application;
- JAR application.

Add the dependency required for developing a Spring-based web/REST application.

After generating the project:

1. Download the generated ZIP file.
2. Extract the project.
3. Open/import it into your IDE.
4. Allow Maven to download the required dependencies.
5. Verify that the initial Spring Boot application starts successfully before continuing.

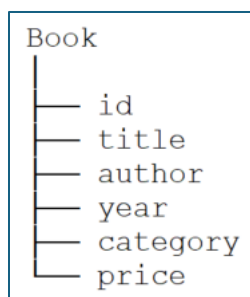
5. Part B — Add the Provided JSON Dataset

Download the **books.json** file attached to the assignment on Moodle.

Place it in the appropriate resources directory:

src/main/resources/books.json

The file contains an **array of book objects**. Each book contains the following information:



Each id uniquely identifies a book.

Important

Do not manually copy all the JSON records into your Java source code. Your application must obtain the book information from the provided **books.json** file.

6. Part C — Create the Book Model

Create a model class named:

Book

The class must represent one object from **books.json**.

Include fields corresponding to:

- **id**
- **title**
- **author**
- **year**
- **category**
- **price**

Provide the Java elements necessary for the JSON data to be converted into **Book** objects.

The goal is to establish the following relationship:

JSON Book Object

↓

Book
Java Object

The field names and data types must be compatible with the provided JSON file.

7. Part D — Create the Service Layer

Create a service class named:

BookService

This component is responsible for working with the book data.

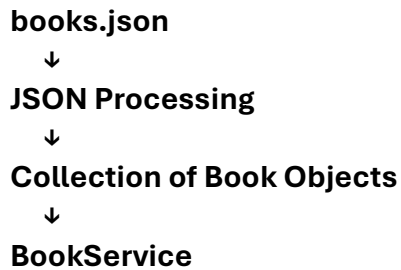
It must:

locate books.json from the application's resources;

1. read the JSON file;
2. convert the JSON array into a collection of Book objects;
3. maintain the collection while the application is running;
4. provide functionality for returning all books;

5. provide functionality for searching for one book by its ID.

Conceptually:



Search Requirement

When the service receives an ID, it must “search” the loaded collection for a book whose:

`book.id == requested ID`

For example, if the requested ID is:

22

the service should search for the book with:

`id = 22`

Practical Requirement

The requested book must be found by searching the loaded book collection. Do not create a series of hard-coded conditions for individual IDs.

8. Part E — Create the REST Controller

Create:

`BookRestController`

Configure it as a Spring REST controller.

Use the following base resource:

`/books`

Your controller must provide **two GET endpoints**.

Endpoint 1 — Retrieve All Books

Implement:

GET /books

This endpoint must return the complete collection of books.

When tested through the client, the response should be a JSON array containing the book objects loaded from books.json.

Endpoint 2 — Retrieve One Book by ID

Implement:

GET /books/{id}

The {id} portion must be obtained dynamically from the URL.

For example:

GET /books/22

means:

Requested ID = 22

The application must search the loaded book collection and return the corresponding book.

The REST controller should delegate the search operation to the service layer rather than implementing the data-loading logic itself.

9. Part F — Handle a Book That Does Not Exist

A REST service should respond appropriately when a requested resource cannot be found.

Test your application using an ID that does not exist in the provided JSON file.

For example:

GET /books/999

Your application must return:

404 Not Found

rather than returning an unrelated book or generating an uncontrolled application error.

Therefore, your application should distinguish between:

Book exists

↓
200 OK + Book JSON

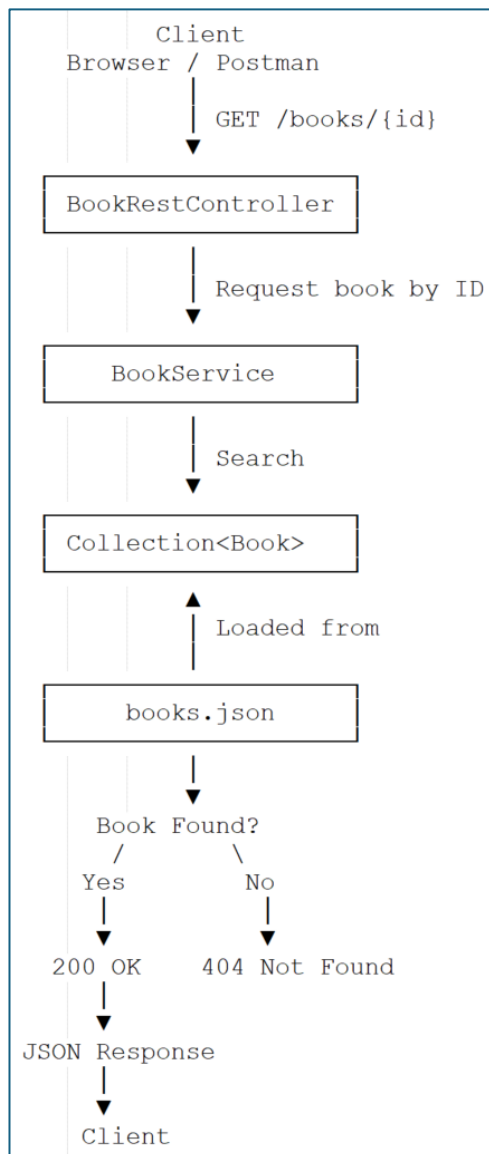
and:

Book does not exist

↓
404 Not Found

10. Expected Application Architecture

Your completed application should conceptually operate as follows:



11. Part G — Test the REST API

Run the Spring Boot application and thoroughly test both endpoints.

At minimum, test the following cases:

Test	Request	Expected Behavior
All books	GET /books	Returns the complete JSON array
Existing book	GET /books/22	Returns the book with ID 22
Another existing book	GET /books/{another valid ID}	Returns the correct book
Invalid ID	GET /books/999	Returns 404 Not Found

You may use a **browser** for simple GET requests, but you should also use **Postman** so that HTTP status codes, headers, and response bodies can be clearly inspected.

12. Practical Report and Screenshot Requirements

This is a **practical programming assignment**. Your report must demonstrate that you actually created, executed, and tested the application.

A report containing only theoretical explanations or copied source code is **not sufficient**.

Your report must include a **comprehensive set of clear screenshots** documenting the development and testing process.

At minimum, include screenshots showing:

1. Spring Initializr project configuration.
2. Required dependency/dependencies selected in Spring Initializr.
3. The completed project structure in your IDE.
4. The relevant Maven configuration in pom.xml.
5. books.json correctly located under src/main/resources.
6. The Book model implementation.
7. The BookService implementation.
8. The BookRestController implementation.
9. The Spring Boot application running successfully.
10. GET /books returning the complete collection.
11. GET /books/22 returning the correct book.
12. A second valid ID returning the correct book.

13. A nonexistent ID producing 404 Not Found.

14. Postman showing the response body and HTTP status for relevant tests.

Each screenshot should be:

- readable;
- appropriately cropped;
- relevant to the requirement being demonstrated;
- accompanied by a short caption explaining what it demonstrates.

Important

Screenshots of source code alone do not prove that the application works. Your report must include evidence of both **implementation and successful execution/testing**.

13. Deliverables

Submit the following through **Moodle**:

1. Your complete Spring Boot project/source code.
2. Your completed report.
3. The comprehensive set of screenshots demonstrating the practical implementation and testing.

Make sure the submitted source code corresponds to the application shown in your screenshots and demonstration.

14. Submission Deadline

You must respect the **due date specified on Moodle**.

It is your responsibility to verify the deadline and ensure that all required files are uploaded successfully before the submission deadline.

15. Mandatory Presentation to the Professor

In addition to submitting your work on Moodle, you must **present and demonstrate the working application to the professor to receive a grade**.

During the presentation, be prepared to:

- start the Spring Boot application;
- explain the project structure;
- show where books.json is stored;
- explain how the JSON data becomes Java Book objects;
- demonstrate GET /books;
- demonstrate GET /books/{id} using an ID selected by the professor;
- demonstrate what happens when an invalid ID is requested;
- explain the responsibilities of the model, service, and controller;
- explain how the ID is obtained from the URL;
- answer basic questions about your implementation.

Mandatory Grading Condition

Submitting the assignment to Moodle without presenting the working application to the professor is not sufficient. If the assignment is not presented to the professor, it will not be graded.

Rubric

Criterion	Description	Weight
Project Setup & Structure	Correct Spring Boot/Maven setup and appropriate model-service-controller organization	15%
JSON Data & Model	Provided JSON file is correctly loaded and mapped to appropriate Book objects	15%
Service Layer	Correctly loads the dataset, returns all books, and searches for books by ID	20%
REST Endpoints	/books and /books/{id} operate correctly using appropriate Spring REST concepts	20%
HTTP & Error Handling	Valid requests return correct data, and nonexistent IDs correctly produce 404 Not Found	10%
Practical Report & Screenshots	Comprehensive, clear evidence of implementation and testing	10%
Live Demonstration & Understanding	Application works during presentation and the student can explain the implementation	10%
Total		100%